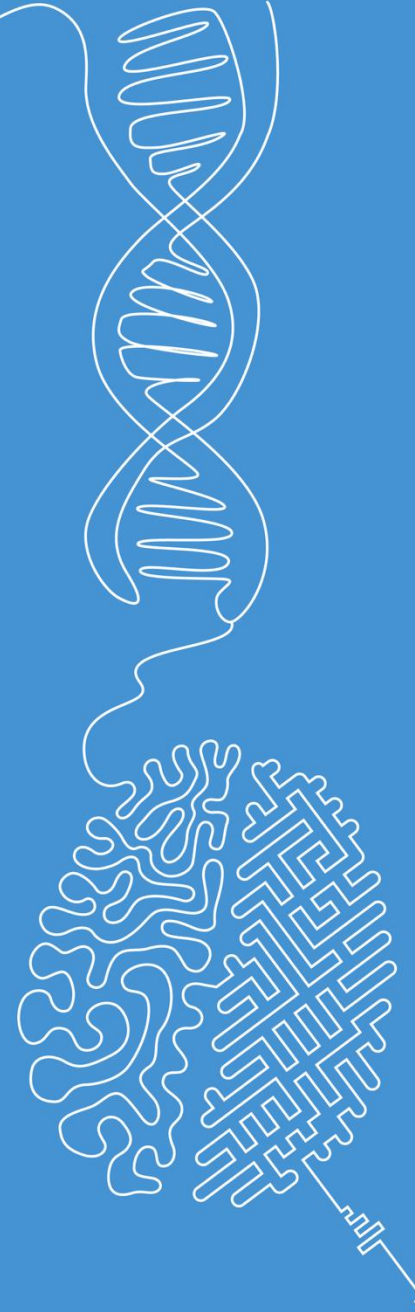


Edge detection (cont.)

Image and Signal Processing

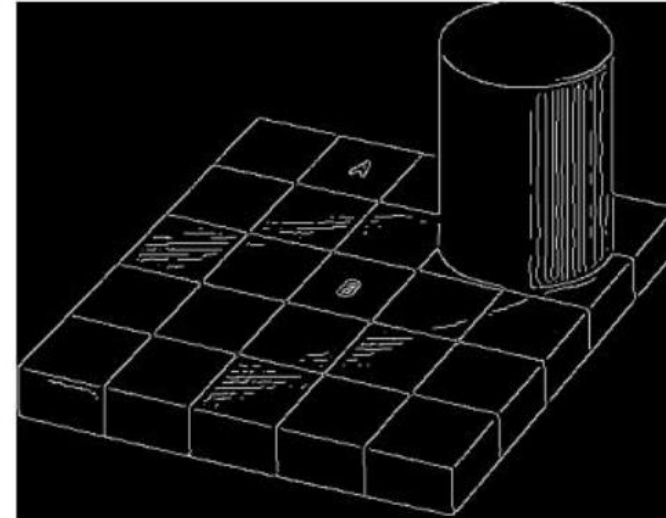
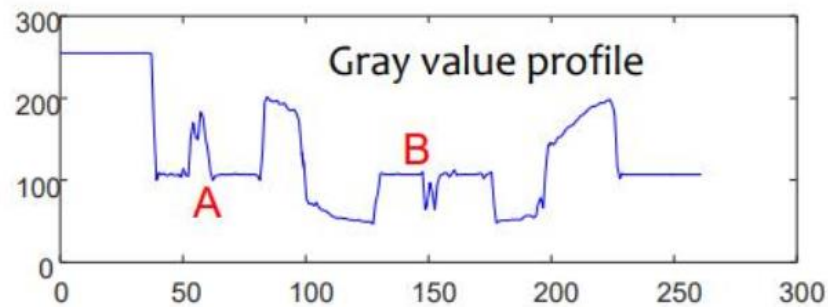
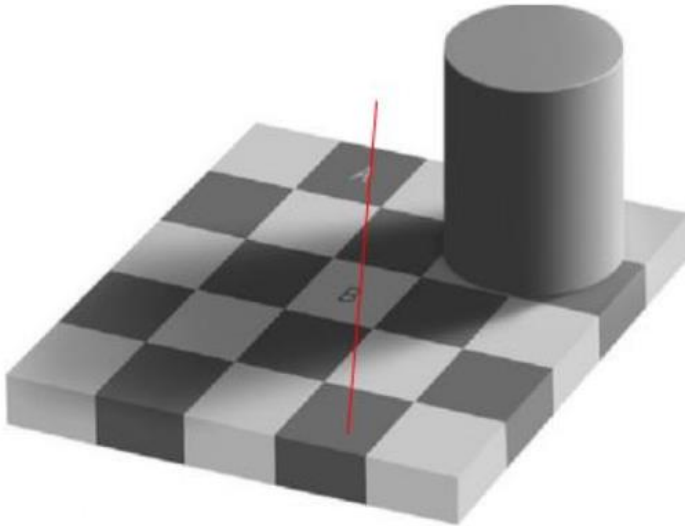
Norman Juchler



Edge detection

continued

Edge detection



Edges contain essential, local image information. Consequently, the detection of edges with edge filters is a fundamental operation in image processing.

Image derivatives

In 1D:



$$B(x, y)$$



First derivative

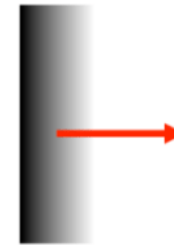
$$\frac{\partial}{\partial x} B(x, y)$$



Second derivative

$$\frac{\partial^2}{\partial x^2} B(x, y)$$

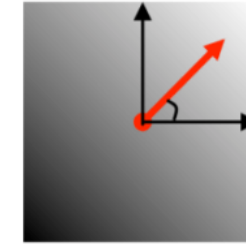
In 2D:



Gradient in x



Gradient in y



Gradient in both x and y

Gradient:

$$\nabla B(x, y) = \begin{pmatrix} \frac{\partial}{\partial x} B(x, y) \\ \frac{\partial}{\partial y} B(x, y) \end{pmatrix}$$

Magnitude:

$$\|\nabla B(x, y)\| = \sqrt{\left(\frac{\partial B}{\partial x}\right)^2 + \left(\frac{\partial B}{\partial y}\right)^2}$$

Orientation:

$$\theta = \arctan \left(\frac{\frac{\partial B}{\partial y}}{\frac{\partial B}{\partial x}} \right)$$

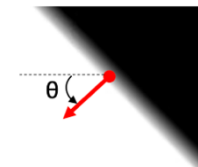
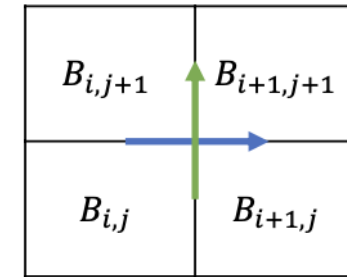


Image derivatives: Finite difference approximations

- Partial derivatives in 2D as finite-difference approximation (for discrete images)

$$\frac{\partial B}{\partial x} \approx ((B_{i+1,j+1} - B_{i,j+1}) + (B_{i+1,j} - B_{i,j}))$$

$$\frac{\partial B}{\partial y} \approx ((B_{i+1,j+1} - B_{i+1,j}) + (B_{i,j+1} - B_{i,j}))$$



Change in x-direction
Change in y-direction

Image

- This can be expressed using filter masks:

$$\frac{\partial B}{\partial x} \approx \begin{bmatrix} -1 & 1 \\ -1 & 1 \end{bmatrix}$$

$$\frac{\partial B}{\partial y} \approx \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix}$$

Note: There are different finite-difference approximations for estimating local derivatives, such as the Prewitt and Sobel operators.

Image derivatives: Finite difference approximations

3x3 Prewittfilter

x-direction

-1	0	1
-1	0	1
-1	0	1

y-direction

1	1	1
0	0	0
-1	-1	-1

Diagonal directions

-1	-1	0
-1	0	1
0	1	1

0	1	1
-1	0	1
-1	-1	0

The increase in filter size allows for **regularization** (noise will be suppressed by taking into account a larger image area).

Image derivatives: Finite difference approximation

3x3 Sobelfilter

x-direction	-1	0	1		1	2	1	y-direction
	-2	0	2		0	0	0	
	-1	0	1		-1	-2	-1	

To further enhance the regularization, the mask elements can be weighted.

Edge filter masks can be constructed for any dimension:

- **Small filter masks** → good localization, sensitive for noise
- **Large filter masks** → bad localization, robust against noise

Edge detection using image gradients

A last step of the gradient method, via a threshold it is decided which pixels belong to an edge and which don't.

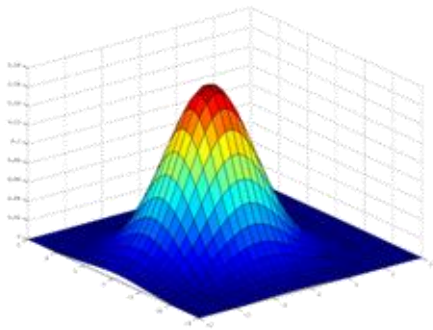
Simple threshold T:

$$\begin{aligned}\|\nabla B(x, y)\| < T & \quad \text{belongs to edge} \\ \|\nabla B(x, y)\| \geq T & \quad \text{does not belong to edge}\end{aligned}$$

Hysteresis based threshold $T_0 < T_1$:

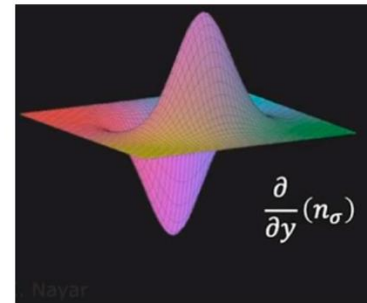
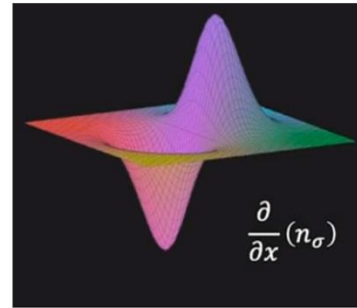
$$\begin{aligned}\|\nabla B(x, y)\| < T_0 & \quad \text{does not belong to edge} \\ \|\nabla B(x, y)\| \geq T_1 & \quad \text{belongs to edge} \\ T_0 \leq \|\nabla B(x, y)\| < T_1 & \quad \text{belongs to edge, if neighbor pixel belongs to edge as well}\end{aligned}$$

Gradient vs. Laplacian



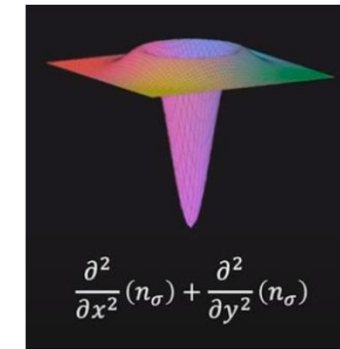
n_σ 2D Gaussian with mean at (0,0), and a fixed standard deviation

Gradient vs. Laplacian



Two values from which the thickness and orientation of the edge can be calculated.

A threshold value is used to determine which voxels/pixels belong to the edge.



One operator that is applied to the image and specifies the location of the edge by zero-value pass-throughs.

Example: Sobel edge detection

Input image
(512x512, uint8)



Gradient magnitude: Sobel
(512x512, float32)



Edges: Sobel
(512x512, bool)



Steps:

1. Smooth input image
2. Apply Sobel filters
3. Compute gradient
4. Apply thresholding

Problems:

- Thick, incomplete, doubled edges

Example: Laplacian edge detection

Input image
(512x512, uint8)



Laplacian filter
(512x512, float32)



Edges: Laplacian
(512x512, bool)



Steps:

1. Smooth input image
2. Apply Laplacian filter (with or without diagonals)
3. Detect zero-crossings

$B_{i-1,j+1}$	$B_{i,j+1}$	$B_{i+1,j+1}$
$B_{i-1,j}$	$B_{i,j}$	$B_{i+1,j}$
$B_{i-1,j-1}$	$B_{i,j-1}$	$B_{i+1,j-1}$

$$\nabla^2 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

detection x- and y-direction

$$\nabla^2 = \begin{bmatrix} 1 & 4 & 1 \\ 4 & -20 & 4 \\ 1 & 4 & 1 \end{bmatrix}$$

detection also in diagonal-direction

Problems:

- Incomplete edges
- Less robust

Example: Canny edge detection

Input image
(512x512, uint8)



Gradient magnitude
(512x512, float32)



Edges: Canny
(512x512, bool)



Steps:

1. Gradient calculation (e.g., using Sobel filters)
2. Non-maximum suppression (→ thin lines)
3. Double threshold (→ strong or weak candidates)
4. Edge tracking by hysteresis

Problems:

- Not robust yet

Smooth the input image!

Example: Canny edge detection (with smoothing)

Input image smoothed
(512x512, uint8)



Gradient magnitude
(512x512, float32)



Edges: Canny
(512x512, bool)



Steps:

1. Gradient calculation (e.g., using Sobel filters)
2. Non-maximum suppression (→ thin lines)
3. Double threshold (→ strong or weak candidates)
4. Edge tracking by hysteresis

Problems:

- Not robust yet

Smooth the input image!

Example: Canny edge detection (detailed)



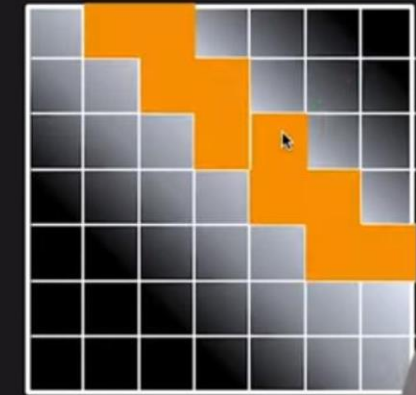
- Smooth Image with 2D Gaussian: $n_{\sigma} * I$
- Compute Image Gradient using Sobel Operator: $\nabla n_{\sigma} * I$
- Find Gradient Magnitude at each pixel: $\|\nabla n_{\sigma} * I\|$
- Find Gradient Orientation at each Pixel:

$$\hat{n} = \frac{\nabla n_{\sigma} * I}{\|\nabla n_{\sigma} * I\|}$$

- Compute Laplacian along the Gradient Direction $\hat{n}$ at each pixel

$$\frac{\partial^2 (n_{\sigma} * I)}{\partial \hat{n}^2}$$

- Find Zero Crossings in Laplacian to find the edge location

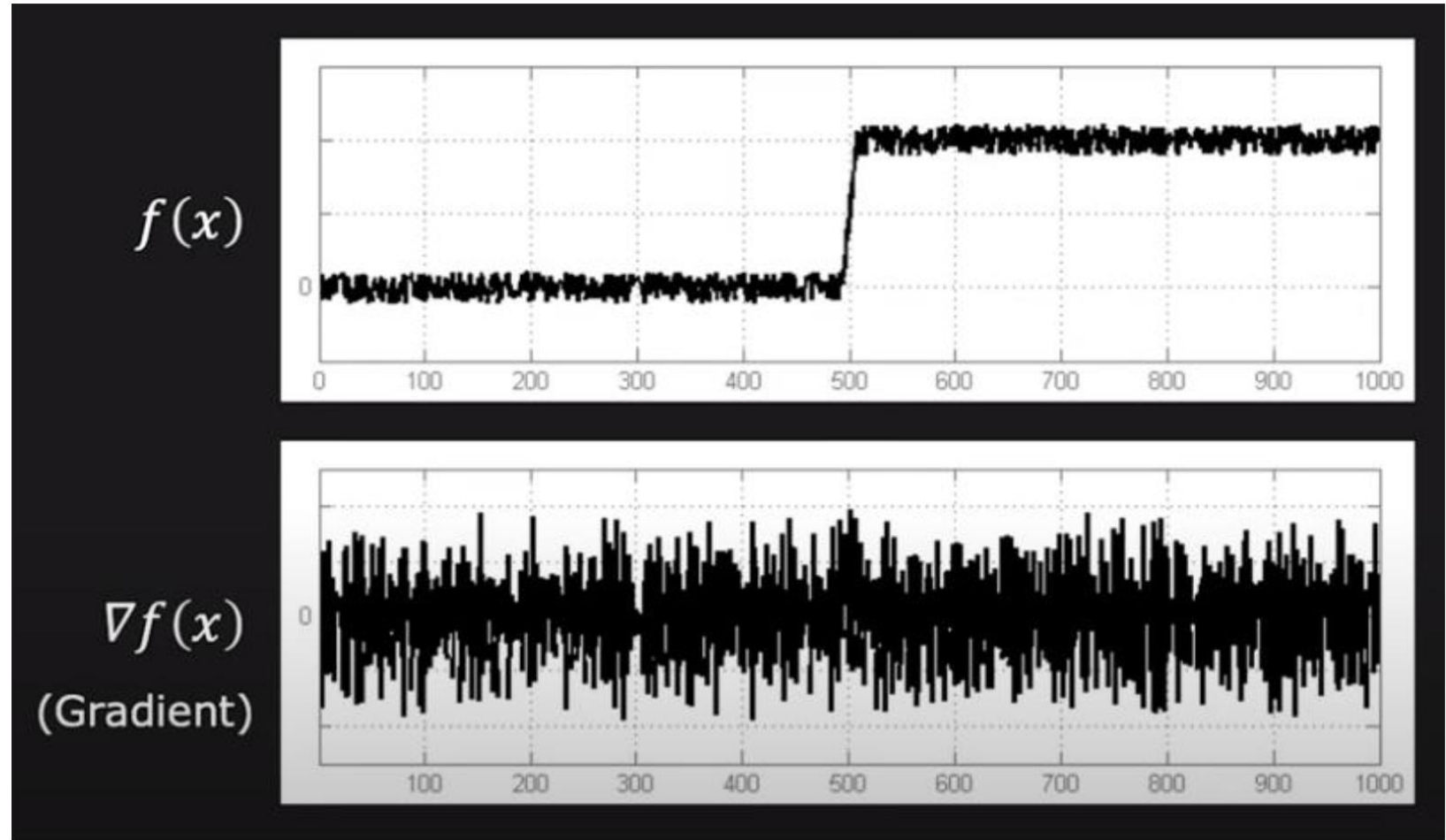


$|\nabla n_{\sigma} * I|$

Edge detection and noise

The noise is amplified when calculating the gradient!

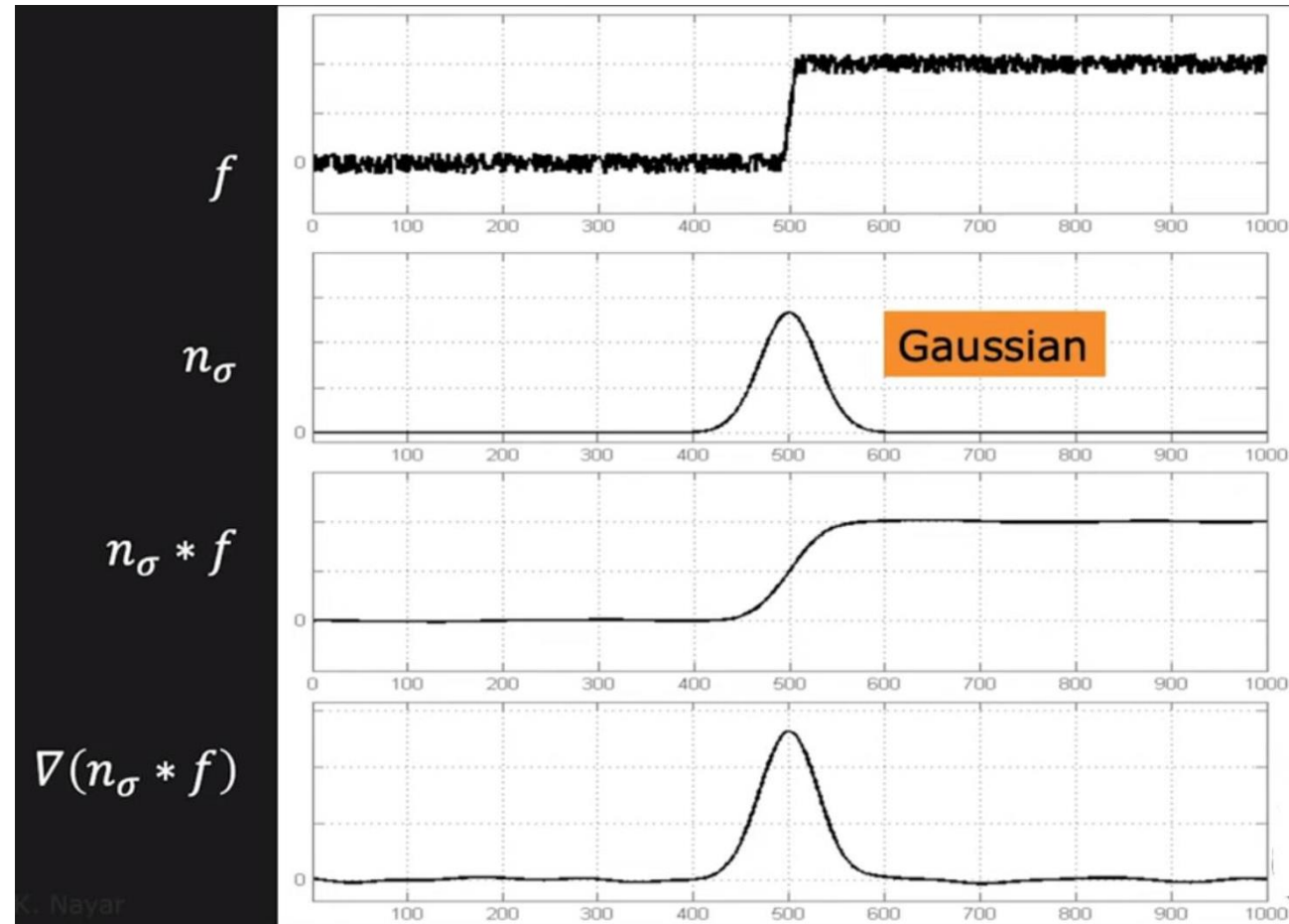
Observation: We need to smooth the input image before computing the gradient!



Edge detection and noise

The noise is amplified when calculating the gradient!

Observation: We need to smooth the input image before computing the gradient!

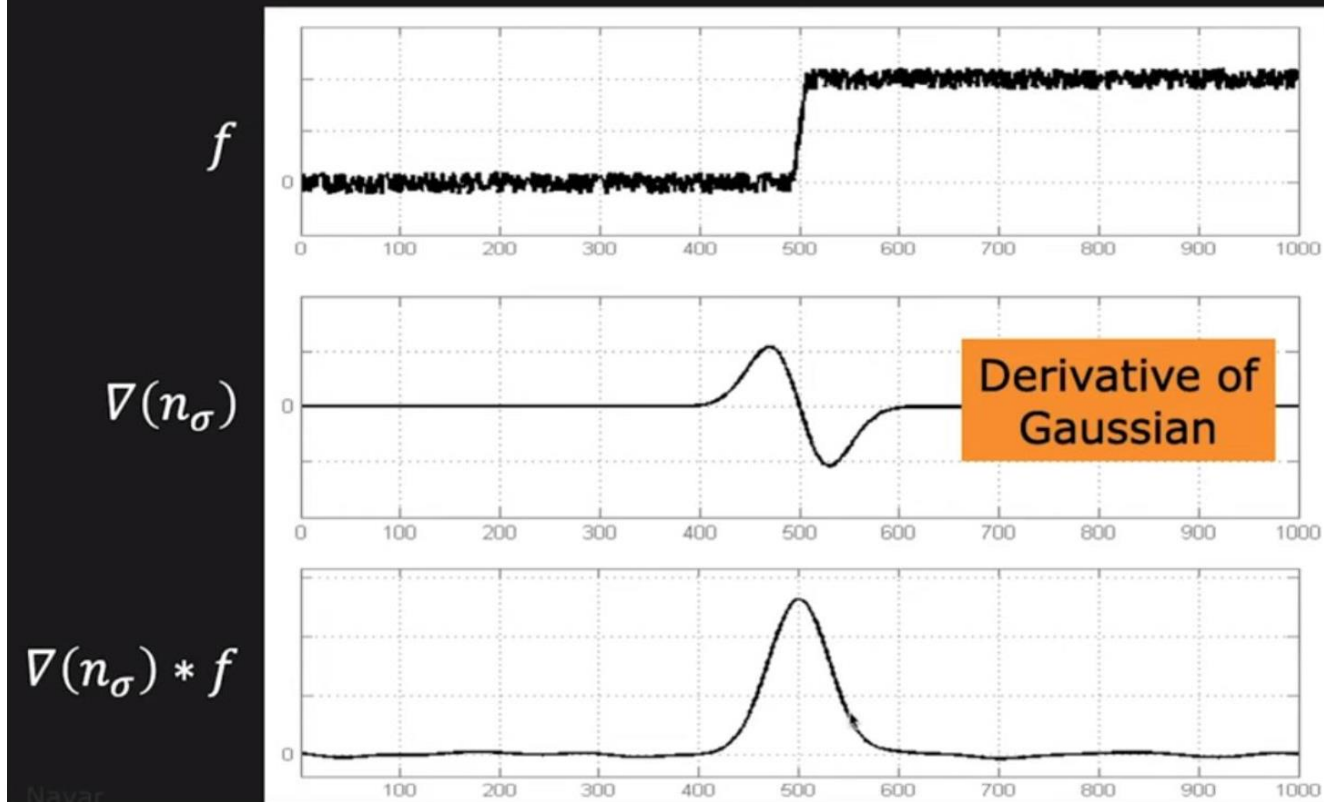


Edge detection and noise

The noise is amplified when calculating the gradient!

Observation: We can combine smoothing and gradient computation!

$$\nabla(n_\sigma * f) = \nabla(n_\sigma) * f \quad \dots \text{saves us one operation.}$$



Edge detection and noise

The noise is amplified when calculating the gradient!

Observation:
Same holds true
for the Laplacian 😊

